

Smart Home Alarm Control System: Concurrent Design and Implementation Report

Eduard Toni Alexandru

September 20, 2026

Abstract

This report outlines the analysis, architectural design, and implementation of a smart home alarm control system built using the Apache Pekko framework. The system manages peripheral sensor networks, user keypad interactions, time delays, and multi-zone safety partitions within a reactive, message-driven concurrent architecture.

1 Problem Analysis

The smart home alarm domain is inherently concurrent, distributed, and reactive. Analyzing the problem reveals several key aspects relevant from a concurrent systems point of view:

- **Autonomous Concurrent Entities:** Peripheral devices (sensors, keypad, siren) and the central control unit operate independently. In a real-world deployment, multiple sensors can detect events simultaneously, and users can interact with the keypad while a sensor trigger message is actively in flight.
- **Asynchronous Communication:** Communication between components cannot rely on blocking calls or shared mutable memory due to race conditions and thread contention. Message passing ensures loose coupling and thread safety.
- **State-Dependent Message Handling:** The system's behavior changes dramatically based on its current operational state (e.g., ignoring sensor triggers when *Disarmed*, but interpreting them as intrusions when *Armed*). Managing this dynamically without explicit locks requires a robust reactive model.
- **Temporal Constraints and Timers:** Exit and entry delays require non-blocking timeout mechanisms that trigger state transitions autonomously without freezing the system execution thread.

1.1 Functional Requirements & System Flow

The alarm control unit reacts dynamically based on the system’s operational states, user commands, and spatial zoning rules:

- **System States:**
 - *Disarmed*: Sensors do not trigger alarms; normal activity is permitted.
 - *Exit Delay*: Triggered after a valid arming request. Sensors remain inactive to let occupants leave. Automatically transitions to *Armed* after a fixed timeout.
 - *Armed*: Sensors are active. Any intrusion event in an active zone triggers the *Entry Delay*.
 - *Entry Delay*: Initiates a countdown. Entering the correct PIN transitions the system to *Disarmed*; otherwise, it escalates to the *Alarm* state.
 - *Alarm (Emergency State)*: Activates the siren. The sole remediation is entering the correct PIN to return to *Disarmed*.
- **User Interaction & Sensors:** Keypad inputs handle arming, disarming, and alarm silencing. Sensors (motion, door/window) report events conditionally based on state.
- **Timing Constraints:** Configurable exit (20–60s) and entry (10–30s) delays govern automatic state timeouts.
- **Zone-Based Control (Bonus Extension):** Sensors are mapped to household zones. Users can perform full arming or partial arming (e.g., Night Mode, keeping bedrooms inactive while securing perimeters).

2 Design Strategy

To address the concurrency and state-management challenges, the system adopts the **Actor Model** via the Apache Pekko framework. The strategy relies on the following core principles:

- **Actor-Based Isolation:** Each system component—the `ControlUnit`, `KeyPad`, individual `Sensor` instances, and the `Siren`—is encapsulated as an autonomous actor. Actors process incoming messages sequentially in their own mailboxes, completely eliminating the need for manual synchronization or synchronization locks.
- **Dynamic State Machine via `getContext().become()`:** Instead of heavy conditional branching (if-else or switch statements checking state variables), the `ControlUnit` dynamically swaps its behavior mapping using Pekko’s `become()` pattern. This cleanly isolates state logic into distinct handlers (*disarmedState*, *exitDelayState*, *armedState*, *entryDelayState*, and *alarmState*).

- **Non-Blocking Timeouts with AbstractActorWithTimers:** Exit and entry delays are managed using Pekko’s built-in single-shot timers. This avoids blocking threads (like `Thread.sleep`) and ensures precise, event-driven timeout handling.
- **Strict Message Protocol:** A strongly typed protocol (`SmartHomeProtocol`) defines all legal interactions, ensuring clear contracts between the user interface, control unit, and peripheral devices.

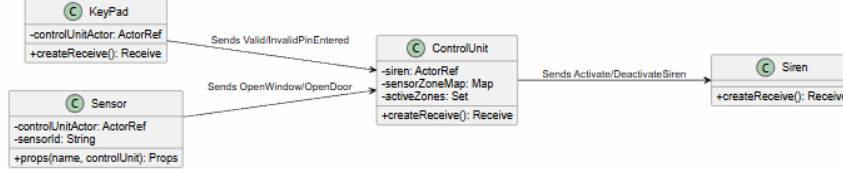


Figure 1: Smart Home Alarm System Actor Class Diagram

3 Implementation Overview

The system implementation is partitioned into modular, testable components:

- **ControlUnit:** Acts as the central state machine. It tracks active zones via an internal map and processes arming requests, sensor triggers, and PIN validations.
- **KeyPad:** Encapsulates user interaction, validating PIN entries against a stored code and translating them into domain messages (`ValidPinEntered` or `InvalidPinEntered`).
- **Sensor:** Represents physical perimeter or motion hardware, forwarding normalized trigger events to the control unit.
- **Siren:** A dedicated actuator responding to activation and deactivation signals during emergency alarm states.
- **Zone-Based Filtering Extension:** During arming requests, users can supply a subset of zones. The `ControlUnit` filters incoming sensor signals against active zones, ignoring breaches in inactive sectors (e.g., night-mode upper floor disarming).

3.1 Conceptual Architecture Overview

The system is built on a decentralized actor model where individual components act as autonomous concurrent entities communicating exclusively via asynchronous messages. The architectural relationships and responsibilities are structured as follows:

- **Keypad Actor (User Interface):**

- *Receives:* `InsertPinMsg` commands representing physical button presses or terminal inputs.
- *Interacts with:* The `ControlUnit` via asynchronous `tell()` messages, translating validation outcomes into `ValidPinEntered` or `InvalidPinEntered` domain events.

- **Sensor Actors (Peripherals):**

- *Receives:* Simulated physical breach events like `OpenWindowMsg` or `OpenDoorMsg`.
- *Interacts with:* The `ControlUnit` by forwarding normalized `SensorTriggeredMsg` events carrying their unique sensor identification string.

- **ControlUnit Actor (Central Brain):**

- *Manages:* System state transitions (*Disarmed*, *ExitDelay*, *Armed*, *EntryDelay*, *Alarm*), temporal timeouts via built-in schedulers, and spatial verification using internal zone-mapping registries.
- *Interacts with:* The `Siren` actor, sending `ActivateSiren` or `DeactivateSiren` signals based on state escalation or successful disarming.

- **Siren Actor (Actuator):**

- *Receives:* Emergency activation and deactivation commands from the control unit to toggle audible alarm outputs.

4 Testing

Testing concurrent and asynchronous actor systems requires specialized tools to handle non-blocking message flows, timeouts, and state verifications.

4.1 Technologies Used

- **Pekko TestKit (TestKit):** Used to spawn test actors, inject messages, and manage the lifecycle of the actor system in an isolated testing sandbox.
- **Test Probes (TestProbe):** Act as mock actor observers to intercept messages sent to external actuators (such as the `Siren`) and assert their delivery.
- **JUnit 5:** Provides testing annotations (`@BeforeEach`, `@AfterEach`, `@Test`) to structure and execute the test suites cleanly.

4.2 Executed Test Scenarios

To ensure thorough coverage of all functional and temporal requirements, the following automated tests were implemented and successfully run:

- **testSirenFiresOnTimeout:** Validates that breaching an active zone during the armed state initiates the entry delay and subsequently fires the siren upon timeout.
- **testSuccessfulDisarmDuringEntryDelay:** Confirms that entering a valid PIN via the keypad during the entry delay successfully disarms the system and prevents the siren from sounding.
- **testPartialArmingIgnoresInactiveZones:** Tests night-mode configurations by verifying that sensors located in unarmred/inactive zones are fully ignored.
- **testSensorsIgnoredWhenDisarmed:** Ensures normal activity logs sensor triggers without escalating to alarms while disarmed.
- **testSensorsIgnoredDuringExitDelay:** Confirms that occupants can exit safely because sensors are inactive during the exit delay countdown.
- **testAlarmStopsWithValidPin & testInvalidPinDuringAlarm:** Verifies that an active alarm state responds correctly to authentication attempts, silencing only upon valid PIN entry while rejecting invalid codes.
- **testFullArmingActivatesAllZones:** Validates that full arming engages all household partitions successfully.

5 Conclusion

The adopted actor-based architecture successfully resolves the concurrent complexities of the smart home alarm system. By utilizing asynchronous message passing, isolated actor states, and non-blocking timers, the implementation achieves high reliability, extensibility, and thread safety without performance bottlenecks.